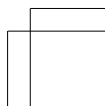
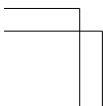
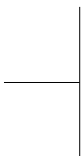


ファイルの変更監視について のほん

y-yagi 著

2025-10-15 版 発行



はじめに

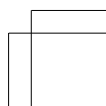
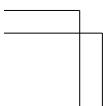
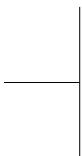
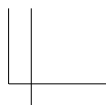
本書の内容

本書は、ファイルシステムイベント監視機能についての本です。きっと皆さんも何らかの形でお使いであろうファイルシステムイベント監視機能について、どのような方法で行われているかについて整理する事を目的としています。

あまり知られていないであろうファイルシステムイベント監視機能について、本書がその理解の一助となれば幸いです。

免責事項

本書に記載された内容は、情報の提供のみを目的としています。したがって、本書を用いた開発、製作、運用は、必ずご自身の責任と判断によって行ってください。これらの情報による開発、製作、運用の結果について、著者はいかなる責任も負いません。また、本書に記載されている情報は、執筆時点でのものです。時間経過によって情報が古くなっている可能性がありますのであらかじめご了承ください。



目次

はじめに	iii
本書の内容	iii
免責事項	iii
第1章 変更監視とは	1
1.1 実装方式	1
ポーリング	2
OS が提供している API	2
1.2 Linux	3
dnotify	3
inotify	4
fanotify	6
コラム: fsnotify 統一バックエンド	7
1.3 *BSD	7
kqueue	7
1.4 macOS	8
FSEvents	9
Endpoint Security	10
1.5 Windows	10
FindFirstChangeNotification	11
	v

目次

	ReadDirectoryChangesW	12
	ReadDirectoryChangesExW	13
	USN ジャーナル	13
1.6	その他	14
第 2 章	様々なライブラリたち	15
2.1	Node.js	15
	chokidar	17
2.2	Go	18
2.3	Rust	18
2.4	Ruby	19
2.5	Python	20
2.6	プログラミング言語に依存しない実装	20
	libuv	20
	watchman	21

第 1 章

変更監視とは

日頃、「ファイルが更新や追加されたら何か処理をする」ということは常日頃発生します。例えば、セキュリティソフト。セキュリティソフトは、ファイルが追加、更新されたら、そのファイルが問題無いか即座にチェックする必要があります。他にも、Dropbox などのファイル同期・バックアップソフトもそうです。ファイルが変更されたら、即座に同期する事を多くのユーザは期待しているでしょう。

このように、ファイルの変更された場合に何か処理をする、というニーズは多くあります。しかし、この変更された場合に何かを行う、という機能がどのように実装されているかの詳細についての記事はあまり多く無いと感じています。この本は、そんな変更監視処理について説明した本です。

1.1 実装方式

変更監視の実装方式は、大きく、ポーリングによる実装と、OS が提供している API を使用した実装の 2 つがあります。それぞれについて見ていきましょう。

ポーリング

特別な API などを使わず、スレッドなどを用いて、監視したいファイルやディレクトリに対して定期的に変更があったかを確認する方式です。

ポーリングのメリットとして、実行される環境に依存せず、同じ機能が提供出来る、という事が挙げられます。後述する OS が提供している API を使用した場合、OS 毎に提供されている機能が事なり、OS 間で挙動の差異が発生してしまう可能性があります。ポーリングだとそのような事はありません。

デメリットとして、変更が正確に把握することが難しい事が挙げられます。例えば、ファイルの更新があったかどうかの間に、ファイルの更新と削除が連続して行われた場合、更新の変更に気付くのが難しくなってしまいます。また、監視するファイルが数ファイルであればどうにかなるかもしれませんが、監視するファイルが複数ディレクトリに跨り数百あるような場合、更新があったかのチェックをするのにリソースを消費してしまい、負荷が掛ってしまう可能性があります。

このような理由により、現在は、OS が提供している API を利用するのが一般的で、何らかの理由によりそれらの API が使用出来ないときの為のセーフティネットとしてポーリングの実装を提供、という形を取るライブラリが多いようです。もし自力でファイル変更監視を行うライブラリの実装を行うような場合は、余程特殊な事情が無い限り、後述する OS が提供する API を使用した実装を行うと良いでしょう。

OS が提供している API

さて、先に説明した通り、ポーリングではファイル変更監視を正確に行うのは難しいです。しかし、昨今の OS は、OS 側でファイル変更監視の為の仕組みが提供されています。一般的にそれらの仕組みは、監視したファイル

やディレクトリを API で指定->指定したファイルやディレクトリで変更があったら変更内容を取得、という事が出来るようになっています。

残念ながら、これらの API は標準化はされておらず、OS 毎に提供されている機能に差異があります。例えば、macOS ではディレクトリを再帰的に監視する為の API が提供されていますが、Linux には同等の API がありません。そのため、OS が提供している API をそのまま使用した場合、提供する機能に差異が発生してしまいます。

それらの差異をどう対処しているかは API を実装しているライブラリ毎に異なります。ライブラリについての話の前に、まず OS が提供している機能について見ていきましょう。

1.2 Linux

まずは Linux について見ていきたいと思います。

dnotify

Linux に初めて導入されたファイルシステムイベント監視の仕組みは、dnotify という機能で Linux 2.4.0 で導入されました。dnotify はディレクトリ自体、もしくはその内部のファイルに変更がかかったときに、ユーザアプリケーションにその変更を通知する事を目的として提供されていました*¹。変更の通知はシグナル送信により行われます。API としては非常にシンプルで、あらかじめ `open(2)` でオープンしておいたディレクトリのファイルディスクリプタに対して `fcntl(2)` の `F_NOTIFY` コマンドを発行し、`DN_ACCESS` や `DN_MODIFY`、`DN_CREATE`、`DN_DELETE`、`DN_RENAME`、`DN_ATTRIB` といったイベント種別をマスクとして渡す、という使い方をします。inotify のように専用の監視インスタンスを作るのではなく、既存のディレクトリのファイルディスクリプタに対して直接登録する仕組みだった点が特徴です。

*¹ dnotify の先頭の"d"は、"directory"の"d"らしいです。

最初に提供された API という事もあり、dnotify には、

- 個別のファイルの監視が行えない。ディレクトリ単位でしか監視できない。
- 通知される情報が少ない (シグナル番号とファイルディスクリプタのみ)。
- 監視するディレクトリ毎にファイルディスクリプタをオープンする必要がある。
- 通知方法にシグナルを使用しているが、シグナルが非同期で送信されるため、競合状態やエラーが発生しやすかった。また、デフォルトで SIGIO を使用していたが、SIGIO はキューイングされないので、通知が欠落する事があった (F_SETSIG でリアルタイムシグナルを指定すれば、`siginfo` からどのファイルディスクリプタの通知かを識別出来て複数の通知をキューイング出来ますが、後述の根本的な問題は解決しません)。
- ハードリンクによって同じファイルに複数のパスが付いている場合、登録したディレクトリ以外を経由した変更は検知出来ない。

等々の問題がありました。そのため、後述する inotify に置き換えられ、現在は使用されなくなっています。

inotify

前述した dnotify の問題を解決する為に登場したのが inotify^{*2}です。inotify は Linux 2.6.13 で導入されました。inotify は、dnotify の問題点を大幅に改善しました。個別のファイルを監視できるようになり、通知される情報も豊富になりました。具体的には、どのファイルで何の変更が発生したかを詳細に知ることができるようになっています。

^{*2} inode-based file event notifications の略。

API は、監視インスタンスを作成する `inotify_init(2)`、監視対象を追加する `inotify_add_watch(2)`、追加した監視を解除する `inotify_rm_watch(2)`、そして通知イベントを受け取る `read(2)` という 4 つのシステムコールで構成されています。`inotify_init(2)` が返すファイルディスクリプタを `read(2)` すると、`struct inotify_event` という構造体（どの監視対象のイベントかを表す `wd`、発生したイベントの種類を表す `mask`、rename 時に `IN_MOVED_FROM` と `IN_MOVED_TO` を対応付けるための `cookie`、対象のファイル名など）が読み出せます。

`inotify` は `inode`^{*3}ベースの通知であり、ディレクトリ単位での再帰的な自動監視は行いません。再帰的に監視したい場合は、利用する側がディレクトリツリーを辿ってそれぞれを監視対象に追加する必要があります。これは意図的な設計判断で、共同設計者の一人である Robert Love によれば、ディレクトリツリーを辿る処理をカーネル内で行うと、ロックを保持したまま長時間かかってしまう可能性があるため、あえてユーザー空間側のライブラリに委ねる事にしたそうです。また、`inotify` にもいくつか制限があります。イベントを発生させたプロセス ID やユーザ ID は取得出来ません。NFS のようなリモートファイルシステムの変更は検知出来ません。また、`mmap(2)` でマップした領域への書き込みのように、ファイル I/O 系のシステムコールを経由しない変更も検知の対象外です。監視対象へのイベント発生ペースがアプリケーション側の処理速度を上回ると、カーネル側のキューが溢れて `IN_Q_OVERFLOW` という特殊なイベントが通知され、それ以降のイベントは失われてしまいます。こうなった場合、アプリケーション側で監視対象を再走査するなどして状態を復元する必要があります。

^{*3} Unix 系ファイルシステムで使われているデータ構造。`inode` はファイルの実体（データブロック）や属性（パーミッション、所有者、タイムスタンプなど）への参照を保持し、ファイル名はその `inode` への参照（ディレクトリエントリ）として扱われます。ファイルシステム内で一意の番号 (`inode` 番号) を持っています。

fanotify

inotify がリリースされた後、さらに高度なファイルシステム監視機能として、fanotify^{*4}が Linux 2.6.36 で導入されました。inotify は、イベント発生源の情報 (プロセス ID やユーザ ID) がとれない、ゲートキーピング機能 gatekeeping^{*5}が無い、NFS のようなリモートファイルシステムのイベントを取得出来ない、等の問題あります。

これは、アンチウイルスソフトのようなセキュリティ関連のアプリケーションで問題になっていたそうです。そこで生まれたのが fanotify です。fanotify は、inotify とは異なるアプローチを取っています。inotify が特定のファイルやディレクトリを監視するのに対し、fanotify はファイルシステム全体を監視することができます。また、先に記載した問題も解消されています。API としては、監視グループを作成する `fanotify_init(2)` と、監視対象を追加する `fanotify_mark(2)` の 2 つのシステムコールで構成されています。`fanotify_mark(2)` では、個別のファイルやディレクトリだけでなく、マウントポイントやファイルシステム全体を監視対象に指定出来ます。これにより、inotify では実現出来なかった、ディレクトリツリーを自前で辿る事なく、レースコンディション無しにファイルシステム全体（またはマウント配下全体）を再帰的に監視する、という事が可能になっています。

なお、ファイルの作成・削除・リネームといったイベントは初期の fanotify では取得出来ず、Linux 5.1 になって初めて対応されました。それまでは、そういった変更を検知したい場合は結局 inotify を併用する必要があります。

^{*4} fscking all notification の略。

^{*5} 操作に介入して許可・拒否等の処理を行うための機能。fanotify では、`FAN_OPEN_PERM` のような「permission イベント」を購読しておくと、対象ファイルへの操作（例えばオープン）が発生した時点でシステムコールがブロックされ、アプリケーションが `fanotify_response` で許可・拒否を応答するまで処理が先に進みません。これにより、ファイルが実際に開かれる前にアンチウイルスソフトが内容をスキャンして拒否する、といった事が可能になっています。

した。

ただし、fanotify を使用するには管理者権限 (CAPSYSADMIN) が必要です。そのため、特別な要件が無い一般的なアプリケーションでは、引き続き inotify が使われる事が多いです。

コラム: fsnotify 統一バックエンド

ここまで紹介した dnotify、inotify、fanotify は、実は fsnotify と呼ばれる共通のカーネル内基盤の上に実装されています*6。fsnotify 自体はユーザー空間から直接呼び出せるシステムコールではなく、あくまでカーネル内部の共通ディスパッチ・登録レイヤーです。dnotify と inotify の実装を整理する事が目的で導入されたわけではなく、実際には fanotify (マウントやファイルシステム全体を監視できる、当時は TALPA と呼ばれていた機能) を実現するために必要になった基盤が Linux 2.6.31 でマージされ、その上に dnotify・inotify も再実装された、という経緯を持っています。

1.3 *BSD

次に、BSD 系 OS について見ていきましょう。

kqueue

BSD 自体にはファイルシステム監視機能はありませんでした。しかし、子孫である FreeBSD には、4.1 から kqueue*7 という機能が追加されました。設計者の Jonathan Lemon が 2001 年の USENIX FREENIX で発表した論文では、スケーラブルである事、柔軟である事、シンプルである事、信頼性が高く正確である事、といった設計目標が掲げられています。kqueue は、

*6 なお、Go 言語の fsnotify ライブラリ (後述) とは名前が同じだけで、直接の関係はありません。

*7 Kernel Queues の略。

元々は I/O 多重化（複数のファイルディスクリプタの状態変化を効率的に監視する）を目的として設計されましたが、ファイルシステムイベントやプロセス、シグナル、タイマーなど、様々な種類のイベントを統一的に扱えるようになっています。API としては、監視キューを作成する `kqueue()` と、監視対象の登録・イベントの取得の両方を担う `kevent()` の 2 つで構成されており、ファイルシステムの変更を監視したい場合は `EVFILT_VNODE` というフィルタを使用します。ファイルシステム以外にも、ソケットの読み書き（`EVFILT_READ`/`EVFILT_WRITE`）やプロセスの状態変化（`EVFILT_PROC`）、シグナル（`EVFILT_SIGNAL`）、タイマー（`EVFILT_TIMER`）など多くのフィルタが用意されており、ファイルシステム監視は `kqueue` が持つ汎用的な仕組みの一用途、という位置づけです。実際、`tail -f` コマンドは、以前はポーリングで実装されていましたが、`kqueue` をサポートする OS では `EVFILT_VNODE` を使う実装に置き換えられています。

ファイルやディレクトリの変更（作成、削除、リネーム、属性変更など）を検知できるようになっていますが、`inotify` 同様、サブディレクトリやファイルを自動で再帰的に監視する機能はありません。

`kqueue` はその後、NetBSD、OpenBSD 及び DragonFly BSD でもサポートされ、BSD 系ではデファクトになりました。

1.4 macOS

続いて macOS について見ていきましょう。macOS が使用している Darwin という OS が BSD をベースにしており、他の BSD 同様に、`kqueue` がサポートされています。それに加えて、macOS では、FSEvents^{*8}という機能も使えるようになっています。次のセクションで詳細を見ていきましょう。

^{*8} File System Events の略。

FSEvents

FSEvents は、macOS で提供されるファイルシステムイベント通知の仕組みです。FSEvents はディレクトリを単位として変更を記録・通知する設計になっており、ディレクトリ階層に対して再帰的に監視が可能です。ディレクトリを単位としているため、ファイル単位の細かなイベントは提供しないようになっています（ただし、macOS 10.7 以降では `kFSEventStreamCreateFlagFileEvents` というフラグを指定する事で、ファイル単位の通知に切り替える事も出来ます。その代わりに、通知されるイベント数は増える事になります）。また、変更をバッファし、ある程度まとめて配信するようになっています。そのため、細かなファイルの変更を検知して向かない仕様になっています。代わりに、ディレクトリ階層に対しては `kqueue` よりも効率的に動作するようになっており、Time Machine のようなファイルシステム全体に対して処理を行う、というような用途に適しています。

アーキテクチャとしては、カーネルで発生した変更が専用のデバイスを経由してユーザー空間の `fseventsd` というデーモンに送られ、そこでフィルタリング・配信が行われます。`fseventsd` は変更履歴を `.fseventsd` というディレクトリ配下に永続化しており、この仕組みにより、アプリケーションは過去のイベント ID を起点にして「自分が起動していなかった間にあった変更」を後から取得する、という事も可能になっています。これは、変更をその場でバッファするだけの `kqueue` には無い、FSEvents 特有の機能です。

なお、大規模なファイル監視ツールである Watchman の開発者による報告では、`git checkout` のような負荷の高い操作の際に、変更の配信が完了した事を保証するはずの `FSEventStreamFlushSync` という API を呼び出した後でも、それ以前の変更に対応するイベントが遅れて届く事があるそうです。Apple は、こういった厳密性が必要な用途向けに、次に紹介する Endpoint Security の使用を推奨しています。

Endpoint Security

Endpoint Security^{*9}は、macOS 10.15 (Catalina) で導入された、セキュリティ監視を目的としたフレームワークです。FSEvents や kqueue がファイルシステムの変更を「後から知る」ための仕組みであるのに対し、Endpoint Security は、ファイル操作を含むシステムコールの発生に介入し、その場で許可・拒否を判断出来る、という点で毛色が異なります。イベントには、システムコールをブロックしアプリケーションからの応答（許可・拒否）を待つ AUTH タイプと、システムコール完了後に通知されるだけの NOTIFY タイプの 2 種類があります。これは Linux の fanotify における、permission イベントと通知イベントの関係に近いものです。AUTH タイプのイベントには応答期限が設けられており、期限内に応答しないクライアントは OS によって強制終了されてしまいます。

利用するには、Apple 発行の専用のエンタイトルメントに加え、ユーザーによるフルディスクアクセスの許可が必要です。fanotify の管理者権限要件よりもさらにハードルが高く、事実上、Apple 公認のセキュリティ製品 (EDR やアンチウイルスソフト等) 向けの機能で、一般的なアプリケーションが単なるファイル監視の目的で使う事は想定されていません。

また、イベントには連番 (seq_num) が付与されており、連番の欠落からドロップしたイベント数を正確に把握出来るようになっています。これは、「何かイベントが失われた」という事しか分からない inotify の IN_Q_OVERFLOW や FSEvents のドロップフラグと比べて、一歩進んだ設計と言えます。

1.5 Windows

Windows は、これまでに 4 つのファイル・ディレクトリ変更監視の為の API を提供してきました。歴史の古い順に、FindFirstChangeNotification、

^{*9} EndpointSecurity.framework として提供されています。

ReadDirectoryChangesW、USN ジャーナル、そして比較的新しい ReadDirectoryChangesExW です。新しい API が登場する度に古い API が廃止された訳ではなく、現在もそれぞれ用途に応じて使い分けられています。

FindFirstChangeNotification

FindFirstChangeNotification は、Windows で最初に提供されたディレクトリ変更監視の API です。Windows 95 時代から利用可能で、比較的シンプルな設計になっています。API にはディレクトリを指定出来るようになっており、指定したディレクトリに変更があった場合通知がされます。ディレクトリの監視はサブディレクトリも含めて再帰的に監視することができます。しかし、通知は、単に変更があったかどうかだけが通知され、どのファイルに変更があったか、などの詳細情報は通知されません。

監視する変更の種類は、dwNotifyFilter 引数でファイル名・ディレクトリ名・属性・サイズ・最終更新日時・セキュリティ情報といった単位で指定します。ただし、サイズや最終更新日時の変更は、ディスクへの書き込みが実際にフラッシュされたタイミングで検知されるため、書き込み直後に即座に通知が来るとは限りません。

API 自体は、変更を待つ為のハンドルを返すだけのシンプルな作りになっています。アプリケーション側は WaitForSingleObject 等でこのハンドルがシグナル状態になるのを待ち、変更を検知したら FindNextChangeNotification を呼び出して再度待受け状態にする、という使い方をします。また、監視対象ディレクトリの内部にあるシンボリックリンクの先で発生した変更は検知出来ません（リンクエントリ自体の作成・削除は検知出来ます）。

現在は、後述する ReadDirectoryChangesW がより詳細な情報を取得出来る上位互換として扱われており、FindFirstChangeNotification が新規の実装で積極的に選ばれる事は少なくなっています。

ReadDirectoryChangesW

ReadDirectoryChangesW^{*10}は、Windows NT 4.0 で導入されたより高機能なディレクトリ変更監視 API です。FindFirstChangeNotification では得られない詳細な変更情報 (変更されたファイル名や変更の内容) を取得出来たり、監視する変更の種類をフィルター出来るようになっています。利用するには、CreateFile に FILE_FLAG_BACKUP_SEMANTICS と FILE_LIST_DIRECTORY を指定して監視対象ディレクトリのハンドルを取得しておく必要があります。フィルタは FindFirstChangeNotification のものに加え、作成日時・最終アクセス日時の変更も指定出来るようになっており、上位互換の関係になっています。

呼び出しは同期・非同期のどちらでも行えます。非同期の場合は、GetOverlappedResult によるポーリング、I/O 完了ポート、completion ルーチン (コールバック) という 3 通りの方法で結果を受け取る事が出来ます。

注意が必要な点として、通知用に確保したバッファが変更の量に対して足りなくなった場合、API は失敗を返さず、成功したように見えつつ書き込まれたバイト数が 0 になる、という形でエラーが表現されます。この場合、その間に発生した変更内容は失われてしまうため、アプリケーション側でディレクトリ全体を再走査する等の対応が必要になります。また、ネットワーク越しに監視する場合は、バッファサイズに 64KB という上限があります。

もう一つの制限として、ファイルが削除された際の通知では、削除されたファイル名しか分からず、そのファイルの属性やサイズといった情報は取得出来ません。この制限を解消した API が、次に紹介する ReadDirectoryChangesExW です。

^{*10} 末尾の "W" までが関数名です。Win32 では、文字列の符号化方法が ANSI 互換文字列の場合末尾が "A"、ワイド文字列の場合 "W" が使われるようになっています。FindFirstChangeNotification については、「FindFirstChangeNotificationA」と「FindFirstChangeNotificationW」の両方が存在しています。

ReadDirectoryChangesExW

ReadDirectoryChangesExW は、Windows 10 バージョン 1709 (Server 2019) で追加された、ReadDirectoryChangesW の派生 API です。基本的なパラメータや挙動は ReadDirectoryChangesW とほぼ同じですが、ReadDirectoryNotifyInformationClass という引数が追加されており、通知の出力フォーマットを、従来の FILE_NOTIFY_INFORMATION と、拡張版の FILE_NOTIFY_EXTENDED_INFORMATION のどちらにするか選択出来るようになっています。

拡張フォーマットを使用すると、削除イベントであってもファイルの属性やサイズといった情報が付与されるようになり、前述した ReadDirectoryChangesW の「削除された物の詳細が分からない」という制限が解消されます。ただし、対応しているファイルシステムやプラットフォームが ReadDirectoryChangesW よりも狭いため、通常は ReadDirectoryChangesW を使い、削除イベントの詳細がどうしても必要な場合にのみ ReadDirectoryChangesExW を検討する、という使い分けが実用的です。

USN ジャーナル

Windows には、USN ジャーナル (Update Sequence Number Journal) というボリュームに加えられた変更の記録を維持する為の機能があります。USN ジャーナルは、NTFS ファイルシステムで提供される低レベルなファイル変更追跡機能で、ファイルやディレクトリに対するあらゆる変更操作を時系列で記録します。当然、ファイルの作成、削除、リネーム等の変更も記録されています。また USN ジャーナルは、API でデータを取得する事が出来、データの取得を行った位置を保存しておくことで、その位置から変更があった情報だけ取得する、ということも出来るようになっています。

コマンドラインからは fsutil usn コマンド (createjournal、readjo

urnal、enumdata 等のサブコマンドを持ちます) で操作出来ます。プログラムから利用する場合は、DeviceIoControl に FSCTL_QUERY_USN_JOURNAL や FSCTL_READ_USN_JOURNAL といった制御コードを渡す事で読み出しを行います。

ファイルシステム全体を監視するため、個別にディレクトリも指定する必要があります。しかし、広域な情報にアクセス出来るため、USN ジャーナルの読み取りには、Administrators グループに所属しているユーザーである事が必須になっています。また、ジャーナル自体の無効化はボリューム全体の MFT を走査する重い処理で、数分掛かる事もあり、同じジャーナルを利用している他のサービスにも影響を与えてしまいます。対応しているファイルシステムも NTFS 3.0 以降と ReFS に限られており、SMB 経由の共有フォルダなどでは利用出来ません。fanotify のように、アンチウイルスソフトのように、管理者権限でファイルシステム全体の変更を取得したい場合にのみ使用され、特別な要件が無い一般的なアプリケーションでは、ReadDirectoryChangesW が使われる事が多いようです。

1.6 その他

ここまで記述した以外にも、Solaris には FEN (File Events Notification、port_create/port_get といった Event Ports ベースの API として提供されています) が、AIX には AHAFS と呼ばれる機能がありますが、本書ではこれらの詳細は割愛させていただきます。

第 2 章

様々なライブラリたち

本章では、どのようなファイル変更監視ライブラリがあるかを、プログラミング言語毎に見ていきたいと思います。後半では、特定のプログラミング言語に依存しないライブラリについても見ていきます。

2.1 Node.js

Node.js は言語本体でファイル変更監視の為に API が提供されています。提供されているのは `fs.watchFile` と `fs.watch` という 2 つの API です。

リスト 2.1: `fs.watchFile`

```
fs.watchFile(filename[, options], listener)
```

リスト 2.2: fs.watch

```
fs.watch(filename[, options][, listener])
```

引数はどちらも同じですね。引数だけ見ると違いがよくわからないのですが、`fs.watchFile`の方はポーリングによる実装になっており、指定出来るのはファイルだけになっています。対して、`fs.watch`はOSが提供しているAPIを使用した実装となっており、こちらはファイル・ディレクトリの両方が指定出来るようになっています。

当然OSが提供しているAPIを使用した方が情報が正確に取得出来る為、公式のドキュメントにも、可能な限り、`fs.watch`の方を使用する事が推奨されています*¹ 前章で説明した通り、OSが提供しているAPIには、OS毎に差異があります。`fs.watch`はAPIの差異の吸収は行わず、OSが提供しているAPIをそのまま使用する、という方針を取っているようです*²。使用しているAPIは下記の通りです。例えば、ファイル名の取得精度です。Linux (inotify)、Windows (ReadDirectoryChangesW)では変更されたファイル名が正確に取得出来ますが、macOS (FSEvents)はディレクトリベースの通知の為、変更されたファイル名が取得出来ない事があります。

なお、上記のような説明があるものの、少しずつ方針が変わっているようです。例えば、以前は再帰的な監視処理がLinuxでは動作しない、という問題がありましたが、現在はこの問題は解消されており*³、影響が大きい差異についてはAPIでOSの差異の吸収を行うようになっているようです。

*¹ https://nodejs.org/docs/latest/api/fs.html#fsfswatchfilefilenameoptions_listener

*² `fs.watch`のドキュメントの注意事項にも、"The fs.watch API is not 100% consistent across platforms, and is unavailable in some situations."との記載があります。

*³ <https://github.com/nodejs/node/pull/45098>

OS	使用している API
Linux	inotify
macOS	kqueue、FSEvents
Windows	ReadDirectoryChangesW

chokidar

先に述べたように、Node.js は言語本体でファイル変更監視の為の API が提供されています。しかし、OS 間で差異がある為、特に古いバージョンではそれだけを使用するのは中々困難だったようです。そのためか、多くの Node.js プロジェクトでは chokidar というライブラリが使用されています。chokidar は、`fs.watch` の上位互換として設計されており、クロスプラットフォームで一貫した動作を提供します。2012 年に、Brunch というビルドツールの一部として切り出される形で誕生しました。

chokidar では、OS 間の差異を吸収し、すべてのプラットフォームで一貫したイベント通知とファイル名の取得を提供し、重複イベントやエラーなどを内部で対象するようになっていきます。監視対象の `FSWatcher` は、`add` (ファイル追加)、`addDir` (ディレクトリ追加)、`change` (変更)、`unlink` (ファイル削除)、`unlinkDir` (ディレクトリ削除)、`ready` (初回スキャン完了) といったイベントを発行し、アプリケーション側はこれらを個別に、あるいは `all` イベントとしてまとめて購読出来ます。

代表的なオプションに `atomic` と `awaitWriteFinish` があります。`atomic` は、多くのエディタが行う「一時ファイルへ書き込んでから元のファイル名にリネームする」という保存方法によって発生する削除・作成イベントの組を、1 つの `change` イベントにまとめてくれる機能です。`awaitWriteFinish` は、ファイルサイズの変化が一定時間止まるまでイベントの発行を保留する機能で、大きなファイルが分割して書き込まれるようなケースに対応する為のものです。

なお、v4 では、macOS 向けにバンドルされていたネイティブの `fsevents` 依存と `glob` サポートが削除され、ネイティブの監視処理は完全に Node.js

の `fs.watch` 任せになっています。

2.2 Go

Go では、`fsnotify/fsnotify` が事実上の標準ライブラリとして使用されています。`fsnotify.NewWatcher()` が `Events` と `Errors` という2つのチャネルを持つ `Watcher` を返し、`watcher.Add(path)` で監視対象のパスを追加する、というシンプルな API になっています。

対応状況は OS によって差があります。Linux (`inotify`)、BSD 系及び macOS (`kqueue`)、Windows (`ReadDirectoryChangesW`)、illumos (`FEN`) は対応済みですが、`fanotify` や macOS ネイティブの `FSEvents`、USN ジャーナル、汎用的なポーリングフォールバックは、本書執筆時点ではまだ実装されていません。また、`chokidar` のようなライブラリ側での再帰監視のサポートは無く、サブディレクトリを監視したい場合は、利用する側でディレクトリツリーを辿って個別に `Add()` を呼び出す必要があります。

macOS では、ネイティブの `FSEvents` ではなく前章で説明した `kqueue` が使われています。`kqueue` は監視対象のファイル毎にファイルディスクリプタを1つ消費するため、多数のファイルを含むディレクトリを監視すると、OS のファイルディスクリプタ数上限に達しやすいという制約を、Go で書かれたアプリケーションもそのまま引き継ぐ事になります。

2.3 Rust

Rust では、`notify-rs/notify` が広く使われています。`rust-analyzer` や `zed`、`watchexec`、`cargo-watch`、`mdBook` といった著名なプロジェクトで採用されています。

対応しているバックエンドは、Linux は `inotify`、Windows は `ReadDirectoryChangesW`、その他の BSD 系向け `kqueue` で、全プラットフォーム向けにポーリングのフォールバックも用意されています。特徴的なのは、macOS

では FSEvents と kqueue のどちらを使うかを Cargo のフィーチャーフラグでビルド時に選択出来るようになっている点です。本書で紹介している他のライブラリは OS 毎にどちらか一方を内部で決め打ちしていますが、notify-rs は利用者側に選択肢を委ねています。

また、デバウンス（短時間に連続する変更をまとめる）機能はコアの `notify` クレートには含まれておらず、`notify-debouncer-mini` や `notify-debouncer-full` という別クレートとして提供されています。chokidar の `awaitWriteFinish` のように標準機能として組み込むのではなく、必要な場合だけ別途取り込む、という設計方針を取っています。

2.4 Ruby

Ruby では、`guard/listen` が広く使われています。

API は `Listen.to(*dirs) { |modified, added, removed| ... }` という形で、採用しているアダプタに関わらず、常に変更・追加・削除された絶対パスの配列 3 つをコールバックに渡すようになっています。監視対象の OS に応じたアダプタは自動的に選択され、実際の処理は別の gem に委譲されます。具体的には、Linux では `rb-inotify`、macOS では `rb-fsevent`、*BSD 系では `rb-kqueue`、Windows では `wdm` が使われ、いずれの環境にも当てはまらない場合はポーリングにフォールバックします。

`:latency`（変更の確認頻度）と `:wait_for_delay`（変更検知後にコールバックを呼ぶまでの遅延時間）を独立して調整出来る他、`:ignore/:only` で監視対象パスを正規表現によりフィルタする事も出来ます。制限事項として、fork したプロセス間ではリスナーが引き継がれない為プロセス毎に `Listen.to` を呼び出す必要がある事や、NFS や Samba、VM の共有フォルダのようにネイティブアダプタが機能しない環境ではポーリングが必須になる事、監視対象が多いプロジェクトでは Linux 環境の `inotify watch` 数上限（`max_user_watches`、カーネルのデフォルト値は環境によって異なる）に達してしまう事などが挙げられます。

2.5 Python

gorakhargosh/watchdog

2.6 プログラミング言語に依存しない実装

libuv

libuv は、Node.js のために開発されたクロスプラットフォームの非同期 I/O ライブラリです。ファイルシステムイベント監視用の機能も提供されており、Node.js の API ではこちらの機能が使われています。当然クラスをプラットフォーム対応が行われており、下記 API が使用されるようになっていきます。

- Linux : inotify
- BSD 系 : kqueue
- macOS : FSEvents、kqueue(使える場合は FSEvents を優先)
- Windows : ReadDirectoryChangesW

API は、監視ハンドルを初期化する `uv_fs_event_init()`、監視を開始する `uv_fs_event_start(handle, cb, path, flags)`、停止する `uv_fs_event_stop()` という構成になっています。再帰監視を有効にする `UV_FS_EVENT_RECURSIVE` フラグは、ネイティブに再帰監視をサポートしている macOS と Windows でのみ実装されており、Linux の inotify 自体には再帰監視の仕組みが無く、libuv 側でもユーザー空間での代替実装は行っていない為、Linux 上では利用出来ません。

Python(`uvloop`)、Rust(`livub-rs`) などのライブラリがあり、Node.js 以外でも使用されています。

第2章 様々なライブラリたち 2.6 プログラミング言語に依存しない実装

watchman

Watchman^{*4}は、Facebook によって開発されたファイル監視ツールです。大規模なコードベースでの効率的なファイル変更検出を目的として設計されており、特にビルドシステムや開発ツールのバックエンドとして使用されています。

特徴

Watchman は以下のような特徴を持っています：

- 高性能な監視：大規模なディレクトリツリーに対して効率的にファイル変更を監視
- クロスプラットフォーム対応：Linux、macOS、Windows で動作
- ユニークなクエリ機能：ファイル変更に対する柔軟なフィルタリングと条件指定が可能
- トリガー機能：特定のファイル変更パターンに対して自動的にコマンドを実行
- JSON 形式のレスポンス：プログラムからの連携が容易

使用している API

Watchman はプラットフォームごとに OS の最適なファイル監視 API を使用します：

OS	使用している API
Linux	inotify
macOS	FSEvents
Windows	ReadDirectoryChangesW

^{*4} <https://github.com/facebook/watchman>

第2章 様々なライブラリたち 2.6 プログラミング言語に依存しない実装

クエリシステム

Watchman の大きな特徴の一つが、強力なクエリシステムです。単純なファイル変更通知だけでなく、以下のような条件でファイルをフィルタリングできます。

- ファイル名のパターンマッチング (glob、正規表現)
- ファイルサイズ、変更時刻による条件指定
- ファイルタイプ (ファイル、ディレクトリ、シンボリックリンク) による絞り込み
- 変更されたファイルのメタデータ (権限、所有者など) 取得

Watchman は、ライブラリとしてアプリケーションに組み込む形ではなく、独立したデーモンプロセスとして動作します。複数のクライアントはソケット経由で同じ watchman プロセスに接続し、監視状態を共有します。例えば `watchman watch <dir>` でディレクトリの監視を開始し、`watchman -- trigger <dir> <name> <glob> -- <cmd>` のようにファイル変更のパターンに対してコマンドを自動実行するトリガーを登録出来ます。これにより、多数の開発ツールが同時にファイル監視を行う環境でも、それぞれが個別に OS の API を呼び出すことなく、システムリソースを効率的に利用出来ます。

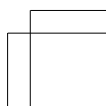
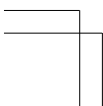
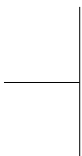
Cookie 機構

Watchman が正確性を担保する為に持っている仕組みが、Cookie と呼ばれる機構です。クエリを実行する際、監視ルート内に一意な名前の一時ファイルを作成し、その作成イベントが通知ストリームに戻ってくるまでクエリの応答を待ちます。これにより、「この Cookie ファイルの作成が見えた時点で、それより前に発生した変更は全て観測済みである」という事を保証出来ます。この仕組みは、大規模な並列テストの実行中に Watchman の処理に遅延が生じ、まだ反映されていない古い状態を返してしまう、という実運

第2章 様々なライブラリたち 2.6 プログラミング言語に依存しない実装

用上の問題への対処として導入されたそうです。

ただし、この保証も万能ではありません。前章で触れた通り、FSEvents には変更の配信完了を保証する仕組みが無く、`git checkout` のような負荷の高い操作の際には、Cookie 機構と組み合わせても、それ以前の変更に対応するイベントが遅れて届いてしまう事があるようです。



ファイルの変更監視についてのほん

2025 年 10 月 15 日 初版第 1 刷 発行

著 者 y-yagi
